

LECTURE NOTES

Data Structures and Algorithm Analysis

Pengantar Pemrograman Berorientasi Obyek

Immanuela Puspasari Saputro, S.Si.,M.T



Learning Outcomes

LO1: Explain the concept of data structures and their usage in Computer Science

Outline Materi

1. Pengenalan *Object-Oriented Programming* (OOP)
2. Enkapsulasi
3. Pewarisan
4. Polimorfisme
5. Abstraksi

2.1

Pengenalan Pemrograman Berbasis Obyek

Pada sesi ini, kita akan memperkenalkan konsep Object-Oriented Programming (OOP) menggunakan bahasa pemrograman Java. OOP adalah paradigma pemrograman yang mengorganisir program dalam bentuk objek. Di Java, OOP merupakan inti dari pengembangan aplikasi yang efisien dan dapat digunakan kembali.

Pemahaman dasar tentang OOP sangat penting karena OOP menjadi paradigma utama dalam pengembangan perangkat lunak modern. Java sendiri adalah bahasa pemrograman yang sangat erat kaitannya dengan OOP, sehingga hampir semua aplikasi yang dibangun menggunakan Java berfokus pada objek.

Dalam *Object-Oriented Programming* (OOP), program dibangun berdasarkan konsep objek, di mana setiap objek mewakili entitas yang memiliki data dan perilaku. Konsep ini mencerminkan bagaimana kita melihat dunia nyata. Sebagai contoh, mobil, buku, dan mahasiswa adalah entitas yang memiliki karakteristik (data) dan tindakan (perilaku). Dalam pemrograman berorientasi objek, kita mencoba merepresentasikan entitas-entitas tersebut dalam bentuk kode program.

Objek dapat diartikan sebagai unit dasar dalam OOP yang memiliki dua komponen penting:

- Atribut (state): data yang dimiliki objek (contoh: warna mobil, nama mahasiswa, saldo rekening).
- Method (behavior): aksi atau perilaku yang dapat dilakukan objek (contoh: mobil dapat berjalan, mahasiswa dapat belajar, rekening dapat melakukan setoran).

Selain objek di dunia nyata, terdapat pula software object, yaitu representasi komputasional dari objek nyata. Software object digunakan untuk memodelkan atribut dan perilaku entitas di dalam program.

OOP didasarkan pada empat konsep utama:

- Encapsulation (Enkapsulasi): Proses menyembunyikan data dari akses langsung dan hanya memberikan akses melalui method tertentu.
- Inheritance (Pewarisan): Kemampuan suatu class untuk mewarisi properti dan method dari class lain.
- Polymorphism (Polimorfisme): Kemampuan objek untuk digunakan dalam berbagai bentuk dan memberikan hasil yang berbeda.
- Abstraction (Abstraksi): Proses menyembunyikan detail implementasi dan hanya menampilkan fungsionalitas penting kepada pengguna.

Mengapa OOP penting?

- Modularitas: Program dapat dibagi menjadi bagian-bagian yang lebih kecil (class dan objek), sehingga memudahkan pemeliharaan dan pengembangan.
- Reuseability: Dengan OOP, kode dapat digunakan kembali (reusability) melalui konsep pewarisan (inheritance).
- Keamanan Data: Melalui enkapsulasi, kita dapat melindungi data dari akses atau modifikasi yang tidak diinginkan.
- Fleksibilitas: Polimorfisme memungkinkan satu interface untuk berbagai implementasi.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS pada sesi 2.

- Pahami perbedaan *software object* dan *real-world object*. Lalu identifikasi empat pilar OOP: Encapsulation, Inheritance, Polymorphism, Abstraction.
- Latihan: Tuliskan contoh objek dunia nyata dan ubah menjadi representasi kode Java (misalnya objek *Mobil* dengan atribut dan method).

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa:

- Diskusikan bagaimana OOP membantu memecah sistem besar menjadi komponen kecil yang saling berinteraksi. Menurut Anda, mengapa konsep objek dan class menjadi pondasi utama dalam bahasa modern seperti Java, Python, dan C#?

2.2

Enkapsulasi

Enkapsulasi (encapsulation) adalah salah satu pilar utama dalam paradigma Object-Oriented Programming (OOP) yang berfungsi untuk melindungi data dan menyembunyikan detail implementasi suatu objek dari akses langsung oleh kode di luar kelasnya. Enkapsulasi melindungi data dalam suatu objek agar tidak diakses atau dimodifikasi secara langsung oleh kode di luar objek. Hal ini dicapai dengan mendeklarasikan atribut sebagai `private` dan menyediakan metode `getter` dan `setter` untuk mengakses atau memodifikasi atribut tersebut. Dengan cara ini, data hanya dapat diubah melalui metode yang terkontrol, sehingga mencegah terjadinya kesalahan atau manipulasi data yang tidak diinginkan. Penggunaan `access modifiers` seperti `private`, `protected`, dan `public` memungkinkan pengembang mengatur tingkat aksesibilitas terhadap atribut dan metode, menjadikan struktur program lebih aman dan terorganisir.

Selain sebagai mekanisme perlindungan data, enkapsulasi juga mendukung modularitas dan `maintainability` dalam pengembangan perangkat lunak. Dengan menyembunyikan detail implementasi di balik antarmuka publik (`public interface`), pengembang dapat memperbarui atau mengubah logika internal suatu kelas tanpa memengaruhi bagian lain dari program yang menggunakan kelas tersebut. Misalnya, ketika struktur data atau cara perhitungan di dalam metode berubah, kode lain tetap berfungsi selama antarmuka (`method name` dan `parameter`) tidak diubah. Dengan demikian, enkapsulasi bukan hanya menjaga integritas data, tetapi juga menjadi dasar penting bagi abstraksi dan isolasi fungsional yang membuat program lebih mudah dikembangkan, diuji, dan dipelihara dalam jangka panjang.

Contoh Encapsulation:

```
class Mahasiswa {  
    private String nama; // atribut private  
    private int umur;  
    // Constructor  
    public Mahasiswa(String nama, int umur) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
    public String getNama() {           // Getter untuk nama  
        return nama;  
    }  
    public void setNama(String nama) {   // Setter untuk nama  
        this.nama = nama;  
    }  
    public int getUmur() {               // Getter untuk umur  
        return umur;  
    }  
    public void setUmur(int umur) {      // Setter untuk umur
```

```
        this.umur = umur;
    }
}

public class Main {
    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa("Andi", 20);
        System.out.println("Nama: " + mhs1.getNama()); // menggunakan getter
        mhs1.setNama("Budi"); // menggunakan setter
        System.out.println("Nama baru: " + mhs1.getNama());
    }
}
```

Penjelasan:

- Atribut nama dan umur dideklarasikan sebagai private, sehingga tidak dapat diakses langsung dari luar class.
- Method `getNama()` dan `setNama()` digunakan untuk membaca dan mengubah nilai nama.
- Enkapsulasi membantu menjaga keamanan data dan memastikan bahwa data hanya dimodifikasi dengan cara yang diinginkan.

Belajar Mandiri



Pelajari bagaimana atribut dilindungi dengan private dan diakses melalui getter dan setter. Jalankan contoh class Mahasiswa dan ubah nilainya menggunakan method `setNama()`. Sebagai Latihan, buat class Produk dengan atribut nama, harga, dan stok yang dienkapsulasi.

2.3

Pewarisan

Inheritance atau pewarisan adalah konsep penting dalam Object-Oriented Programming (OOP) yang memungkinkan sebuah kelas baru (subclass atau child class) mewarisi atribut dan metode dari kelas lain yang sudah ada (superclass atau parent class). Dengan pewarisan, kita dapat membangun hierarki kelas yang merepresentasikan hubungan *is-a* antarobjek, misalnya Mobil adalah turunan dari Kendaraan. Konsep ini mempermudah pengelolaan kode karena kita dapat menggunakan kembali (*reusability*) logika dan perilaku yang sudah didefinisikan di superclass tanpa perlu menulis ulang kode yang sama di setiap subclass. Selain itu, subclass dapat menambahkan atribut atau metode baru untuk memperluas fungsionalitas, atau melakukan overriding terhadap metode dari superclass untuk menyesuaikan perilaku yang lebih spesifik.

Java mengimplementasikan pewarisan menggunakan kata kunci `extends`. Misalnya, class `Mobil` `extends` `Kendaraan` berarti `Mobil` mewarisi seluruh atribut dan metode dari `Kendaraan`. Mekanisme ini tidak hanya mempercepat proses pengembangan, tetapi juga meningkatkan konsistensi program karena perubahan pada superclass dapat berdampak langsung pada semua subclass yang terkait. Selain itu, Java mendukung pewarisan bertingkat (*multilevel inheritance*), di mana sebuah subclass juga dapat berperan sebagai superclass bagi kelas lainnya. Namun, Java membatasi pewarisan ganda antar kelas untuk menghindari ambiguitas, dan sebagai gantinya menyediakan interface untuk mendukung *multiple inheritance* dalam bentuk perilaku. Dengan pewarisan yang tepat, program menjadi lebih modular, efisien, dan mudah diperluas, karena hubungan antarobjek mencerminkan struktur hierarki dunia nyata.

Contoh Inheritance:

```
// Superclass
class Kendaraan {
    String merk;
    int tahun;
    // Constructor
    public Kendaraan(String merk, int tahun) {
        this.merk = merk;
        this.tahun = tahun;
    }
    public void tampilkanInfo() {
        System.out.println("Merk: " + merk);
        System.out.println("Tahun: " + tahun);
    }
}

// Subclass yang mewarisi dari Kendaraan
class Mobil extends Kendaraan {
    int jumlahPintu;
```

```
public Mobil(String merk, int tahun, int jumlahPintu) {  
    super(merk, tahun); // memanggil constructor superclass  
    this.jumlahPintu = jumlahPintu;  
}  
  
public void tampilkanInfoMobil() {  
    tampilkanInfo(); // memanggil method dari superclass  
    System.out.println("Jumlah Pintu: " + jumlahPintu);  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil("Toyota", 2020, 4);  
        mobil1.tampilkanInfoMobil();  
    }  
}
```

Penjelasan:

- Class Mobil adalah subclass yang mewarisi properti dan method dari class Kendaraan.
- `super(merk, tahun)` digunakan untuk memanggil constructor dari superclass.
- Mobil menambahkan properti baru yaitu `jumlahPintu` dan method baru `tampilkanInfoMobil()`.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS pada sesi 2

- Pelajari bagaimana class turunan mewarisi atribut dan method dari class induk. Jalankan contoh class Mobil yang mewarisi dari Kendaraan. Sebagai latihan: buat class Pegawai sebagai superclass dan dosen sebagai subclass dengan tambahan atribut matakuliah.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa:

- Berikan contoh nyata bagaimana enkapsulasi melindungi data penting dalam program, dan bagaimana inheritance menghemat waktu dengan penggunaan kembali kode. Ceritakan pengalaman Anda membuat class turunan atau menggunakan getter–setter

Apakah Kamu Tahu



Mengapa Java hanya mendukung **single inheritance** untuk class tetapi **multiple inheritance** untuk interface?

2.4

Polimorfisme

Polymorphism dalam pemrograman berorientasi objek (OOP) berasal dari kata Yunani yang berarti "banyak bentuk." Dalam konteks pemrograman, polimorfisme memungkinkan objek-objek dari berbagai class untuk diperlakukan seolah-olah mereka adalah objek dari class yang sama. Dengan kata lain, polimorfisme memungkinkan satu method atau tindakan bekerja dengan cara yang berbeda tergantung pada objek yang memanggilnya.

Polimorfisme memungkinkan objek untuk diakses melalui referensi dari class induk atau interface. Dalam polimorfisme, method yang sama dapat berperilaku berbeda berdasarkan objek mana yang memanggilnya.

Ada dua jenis polimorfisme:

- **Compile-time Polymorphism (Method Overloading):** Terjadi ketika beberapa method memiliki nama yang sama tetapi parameter yang berbeda.
- **Runtime Polymorphism (Method Overriding):** Terjadi ketika subclass menyediakan implementasi spesifik untuk method yang ada di superclass

Compile-time Polymorphism (Method Overloading)

Polimorfisme waktu kompilasi terjadi ketika beberapa method memiliki nama yang sama tetapi memiliki parameter yang berbeda (baik dalam jumlah parameter atau tipe data parameter). Ini dikenal sebagai method overloading. Kompiler Java akan memutuskan method mana yang akan dipanggil berdasarkan tanda tangan method (parameter yang diberikan saat pemanggilan method).

Contoh Method Overloading:

```
class Kalkulator {  
    // Method untuk menjumlahkan dua integer  
    public int tambah(int a, int b) {  
        return a + b;  
    }  
    // Method untuk menjumlahkan tiga integer  
    public int tambah(int a, int b, int c) {  
        return a + b + c;  
    }  
    // Method untuk menjumlahkan dua bilangan desimal (double)  
    public double tambah(double a, double b) {  
        return a + b;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Kalkulator calc = new Kalkulator();  
  
        // Memanggil method yang menjumlahkan dua integer  
        System.out.println("Hasil 2 bilangan int: " + calc.tambah(3, 4));  
  
        // Memanggil method yang menjumlahkan tiga integer  
        System.out.println("Hasil 3 bilangan int: " + calc.tambah(3, 4, 5));  
  
        // Memanggil method yang menjumlahkan dua bilangan desimal  
        System.out.println("Hasil 2 bilangan double: " + calc.tambah(3.5, 4.2));  
    }  
}
```

Penjelasan:

- Dalam contoh di atas, terdapat tiga method tambah yang memiliki nama yang sama tetapi tanda tangan method (jumlah dan tipe parameter) berbeda.
- Kompiler akan memilih method mana yang akan dipanggil berdasarkan tipe data dan jumlah argumen yang dikirimkan ke method

Runtime Polymorphism (Method Overriding)

Polimorfisme waktu jalan terjadi ketika subclass memiliki implementasi khusus untuk method yang sudah ada di superclass. Ini dikenal sebagai method overriding. Polimorfisme ini bekerja saat runtime, artinya Java akan menentukan method mana yang dipanggil berdasarkan jenis objek sebenarnya, meskipun objek tersebut direferensikan menggunakan tipe superclass.

Contoh Method Overriding:

```
class Hewan {  
    // Method yang akan di-override di subclass  
    public void bersuara() {  
        System.out.println("Hewan bersuara");  
    }  
}  
  
class Kucing extends Hewan {  
    // Override method bersuara  
    @Override  
    public void bersuara() {  
        System.out.println("Kucing mengeong");  
    }  
}
```

```
}  
}  
class Anjing extends Hewan {  
    // Override method bersuara  
    @Override  
    public void bersuara() {  
        System.out.println("Anjing menggonggong");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Hewan hewan1 = new Kucing(); // Referensi Hewan, objek Kucing  
        Hewan hewan2 = new Anjing();  // Referensi Hewan, objek Anjing  
  
        // Polimorfisme terjadi saat runtime  
        hewan1.bersuara(); // Output: Kucing mengeong  
        hewan2.bersuara(); // Output: Anjing menggonggong  
    }  
}
```

Penjelasan:

- Class Hewan memiliki method bersuara() yang di-override oleh class Kucing dan Anjing.
- Meskipun objek hewan1 dan hewan2 direferensikan sebagai Hewan, mereka menjalankan implementasi bersuara() dari class Kucing dan Anjing, karena jenis objek yang sebenarnya adalah Kucing dan Anjing.
- Polimorfisme runtime menentukan bahwa method yang dipanggil adalah milik objek yang sebenarnya (bukan tipe referensinya).

Polimorfisme sangat penting dalam OOP karena memungkinkan kode untuk lebih fleksibel, dinamis, dan mudah dipelihara. Dengan polimorfisme, kita dapat:

1. Menulis kode yang lebih generik. Anda bisa membuat referensi terhadap class induk, sementara objek yang direferensikan bisa berasal dari subclass yang berbeda.
2. Mempermudah pengelolaan kode, terutama dalam hal open/closed principle, di mana class dapat diperluas tanpa perlu mengubah kode yang sudah ada.
3. Memanfaatkan kemampuan overriding untuk memberikan perilaku yang berbeda pada method yang sama di class yang berbeda.

Contoh Implementasi Polimorfisme dalam Kehidupan Nyata

Bayangkan sebuah skenario di mana Anda memiliki sebuah Remote yang bisa mengontrol berbagai perangkat seperti TV, AC, atau DVD Player. Anda bisa menggunakan tombol yang sama pada remote untuk menghidupkan perangkat yang berbeda. Namun, meskipun tombol yang sama digunakan, perangkat yang dihidupkan akan berbeda tergantung perangkat apa yang terhubung. Ini adalah contoh nyata dari polimorfisme.

Dalam OOP, kita bisa merepresentasikan perangkat-perangkat ini sebagai subclass (TV, AC, DVDPlayer) dari class induk (PerangkatElektronik), dan method `hidupkan()` bisa diimplementasikan secara berbeda untuk setiap perangkat.

Contoh Implementasi dalam Java:

```
class PerangkatElektronik {
    // Method yang akan di-override
    public void hidupkan() {
        System.out.println("Menghidupkan perangkat elektronik");
    }
}

class TV extends PerangkatElektronik {
    @Override
    public void hidupkan() {
        System.out.println("Menghidupkan TV");
    }
}

class AC extends PerangkatElektronik {
    @Override
    public void hidupkan() {
        System.out.println("Menghidupkan AC");
    }
}

public class Main {
    public static void main(String[] args) {
        // Menggunakan polimorfisme
        PerangkatElektronik perangkat1 = new TV();
        PerangkatElektronik perangkat2 = new AC();

        perangkat1.hidupkan(); // Output: Menghidupkan TV
    }
}
```

```
        perangkat2.hidupkan(); // Output: Menghidupkan AC
    }
}
```

Dalam contoh ini, meskipun kita menggunakan referensi PerangkatElektronik, method yang dijalankan berbeda-beda tergantung pada objek yang sebenarnya (TV atau AC), yang merupakan bentuk nyata dari polimorfisme.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS pada sesi 2

- Pahami dua bentuk polimorfisme: compile-time (method overloading) dan runtime (method overriding). Jalankan contoh class Kalkulator (overloading) dan Hewan–Kucing–Anjing (overriding).
- Implementasikan method hitung() dengan dua versi berbeda (menjumlah dan mengalikan).

2.5

Abstraksi

Abstraksi adalah konsep dalam Object-Oriented Programming (OOP) yang bertujuan untuk menyembunyikan detail implementasi yang tidak diperlukan dan hanya menampilkan fungsionalitas penting kepada pengguna. Dalam kata lain, abstraksi membantu kita fokus pada apa yang dilakukan oleh suatu objek, bukan bagaimana objek tersebut melakukannya.

Bayangkan Anda menggunakan mesin ATM. Anda hanya perlu tahu cara memasukkan kartu, memasukkan PIN, dan mengambil uang. Anda tidak perlu tahu bagaimana mesin ATM bekerja di dalam untuk memproses transaksi Anda. Ini adalah contoh nyata dari abstraksi. Abstraksi adalah proses menyembunyikan detail implementasi dari pengguna dan hanya menampilkan fungsionalitas yang penting. Ini dapat dicapai dengan menggunakan abstract class atau interface.

- Abstract class adalah class yang tidak bisa diinstansiasi dan bisa memiliki method abstract (tanpa implementasi).
- Interface adalah blueprint dari class yang hanya berisi deklarasi method tanpa implementasi.

Abstract Class (Kelas Abstrak)

Abstract class adalah class yang tidak dapat diinstansiasi, artinya Anda tidak bisa membuat objek dari class tersebut. Abstract class biasanya digunakan sebagai blueprint atau kerangka untuk class-class turunan (subclass). Abstract class dapat memiliki method dengan atau tanpa implementasi.

- Method abstract adalah method yang hanya dideklarasikan tanpa implementasi. Subclass yang mewarisi abstract class harus mengimplementasikan method abstract tersebut.
- Method non-abstract adalah method yang sudah memiliki implementasi di dalam abstract class, dan subclass tidak diwajibkan untuk mengubahnya kecuali diperlukan.

Contoh Abstract Class dalam Java:

```
// Abstract class
abstract class Bentuk {
    // Abstract method (tanpa implementasi)
    abstract void gambar();

    // Non-abstract method (dengan implementasi)
    public void deskripsi() {
        System.out.println("Ini adalah bentuk geometris.");
    }
}

// Subclass mengimplementasikan abstract method
class Lingkaran extends Bentuk {
    @Override
    void gambar() {
```

```
        System.out.println("Menggambar lingkaran.");
    }
}
// Subclass mengimplementasikan abstract method
class Persegi extends Bentuk {
    @Override
    void gambar() {
        System.out.println("Menggambar persegi.");
    }
}
public class Main {
    public static void main(String[] args) {
        Bentuk bentuk1 = new Lingkaran(); // Menggunakan polimorfisme
        Bentuk bentuk2 = new Persegi();
        bentuk1.gambar(); // Output: Menggambar lingkaran.
        bentuk2.gambar(); // Output: Menggambar persegi.
        bentuk1.deskripsi(); // Output: Ini adalah bentuk geometris.
    }
}
```

Penjelasan:

1. Abstract class `Bentuk` tidak dapat diinstansiasi langsung, tetapi bisa menjadi referensi untuk objek subclass (`Lingkaran` dan `Persegi`).
2. Method `gambar()` adalah abstract method yang harus diimplementasikan oleh subclass (`Lingkaran` dan `Persegi`).
3. Method `deskripsi()` adalah method non-abstract dengan implementasi yang bisa dipanggil oleh subclass.

Keuntungan Penggunaan Abstract Class

1. Abstract class berguna ketika Anda ingin memberikan kerangka dasar untuk subclass tetapi masih memberi kebebasan pada subclass untuk mendefinisikan detail implementasinya.
2. Abstract class dapat memiliki method dengan atau tanpa implementasi, sehingga memungkinkan untuk menyediakan beberapa fitur dasar yang bisa digunakan oleh subclass.

Interface

Interface adalah bentuk lain dari abstraksi. Interface berisi deklarasi method tanpa implementasi (semua method dalam interface bersifat abstract secara default). Tidak seperti abstract class, sebuah class dapat mengimplementasikan beberapa interface sekaligus, sehingga interface sering digunakan untuk mendefinisikan perilaku yang harus dimiliki oleh beberapa class yang berbeda.

- Dalam interface, semua method bersifat public dan abstract.

Interface dapat mendefinisikan kontrak atau blueprint untuk class tanpa harus mengimplementasikan detailnya

Contoh Interface dalam Java:

```
// Interface
interface Hewan {
    void bersuara(); // Method abstract
    void bergerak(); // Method abstract
}

// Implementasi interface di class Kucing
class Kucing implements Hewan {
    @Override
    public void bersuara() {
        System.out.println("Kucing mengeong.");
    }

    @Override
    public void bergerak() {
        System.out.println("Kucing berjalan dengan 4 kaki.");
    }
}

// Implementasi interface di class Burung
class Burung implements Hewan {
    @Override
    public void bersuara() {
        System.out.println("Burung berkicau.");
    }

    @Override
    public void bergerak() {
        System.out.println("Burung terbang.");
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Hewan kucing = new Kucing(); // Polimorfisme
        Hewan burung = new Burung();

        kucing.bersuara(); // Output: Kucing mengeong.
        kucing.bergerak(); // Output: Kucing berjalan dengan 4 kaki.
        burung.bersuara(); // Output: Burung berkicau.
        burung.bergerak(); // Output: Burung terbang.
    }
}

```

Penjelasan:

1. Interface Hewan mendeklarasikan dua method (bersuara() dan bergerak()) tanpa implementasi.
2. Class Kucing dan Burung mengimplementasikan interface Hewan dan memberikan implementasi khusus untuk setiap method.
3. Setiap class yang mengimplementasikan interface harus mengimplementasikan seluruh method yang dideklarasikan di dalam interface.

Keuntungan Penggunaan Interface

1. Interface memungkinkan multiple inheritance di Java karena satu class dapat mengimplementasikan beberapa interface. Ini mengatasi keterbatasan pewarisan tunggal pada class.
2. Interface mendefinisikan kontrak atau aturan yang harus diikuti oleh class yang mengimplementasikannya, yang memastikan konsistensi di seluruh program.

Perbandingan Antara Abstract Class dan Interface

Abstract Class	Interface
Bisa memiliki method dengan implementasi (non-abstract method).	Hanya memiliki method tanpa implementasi (abstract method).
Bisa memiliki atribut dan method yang dilindungi (protected).	Hanya bisa memiliki public method dan public static final atribut.
Sebuah class bisa mewarisi satu abstract class.	Sebuah class bisa mengimplementasikan lebih dari satu interface.
Cocok digunakan saat class memiliki hubungan hierarkis (contoh: Bentuk dan Lingkaran).	Cocok digunakan saat tidak ada hubungan hierarkis tetapi membutuhkan perilaku umum (contoh: Hewan, Kendaraan).

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS pada sesi 2

- Pelajari perbedaan *abstract class* dan *interface*. Jalankan contoh class Bentuk, Lingkaran, dan Persegi serta interface Hewan
- Latihan: Buat interface Pembayaran dengan method proses() dan implementasinya pada class EWallet dan KartuKredit.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

- Banyak mahasiswa memahami polimorfisme hanya sebagai “method sama, hasil berbeda.” Coba jelaskan contoh dunia nyata yang menggambarkan konsep ini, serta bagaimana abstraksi membantu kita menyembunyikan kompleksitas sistem tanpa kehilangan fungsionalitas utama.

Evaluasi Individu



Kerjakan indivisual assignment di LMS BINUS ONLINE pada sesi 2

Post-Test



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

1. Apa tujuan utama dari enkapsulasi dalam OOP?
 - A. Menyembunyikan detail data dan melindunginya dari akses langsung
 - B. Meningkatkan kecepatan eksekusi program
 - C. Membuat method dapat dipanggil dari luar class
 - D. Menghubungkan dua class berbedaJawaban: A
2. Manakah pernyataan yang benar tentang inheritance di Java?
 - A. Subclass tidak dapat mengakses method superclass
 - B. Subclass dapat menggunakan kembali atribut dan method superclass
 - C. Java tidak mendukung inheritance
 - D. Superclass mewarisi method dari subclassJawaban: B
3. Apa yang dimaksud dengan method overriding?
 - A. Menulis ulang method dengan nama dan parameter berbeda
 - B. Menghapus method dari superclass
 - C. Menulis ulang method superclass dengan nama dan parameter yang sama di subclass
 - D. Membuat dua method dengan nama sama di class yang samaJawaban: C
4. Apa perbedaan mendasar antara abstract class dan interface di Java?
 - A. Abstract class tidak bisa memiliki method dengan implementasi
 - B. Interface dapat memiliki konstruktor
 - C. Abstract class bisa memiliki method dengan atau tanpa implementasi, sedangkan interface hanya berisi deklarasi method
 - D. Interface tidak bisa digunakan sebagai tipe referensiJawaban: C
5. Polimorfisme memungkinkan...
 - A. Setiap class memiliki atribut yang unik
 - B. Semua method harus bersifat abstract
 - C. Kelas anak tidak dapat mengubah perilaku induknya
 - D. Method yang sama memiliki perilaku berbeda tergantung objek yang memanggilnyaJawaban: D

Kesimpulan

Pada materi ini, kita mempelajari konsep Object-Oriented Programming (OOP), yaitu paradigma pemrograman yang berfokus pada objek. Objek-objek ini mewakili entitas nyata yang memiliki data (disebut atribut) dan perilaku (disebut metode). Konsep ini membantu kita untuk menulis kode yang lebih terstruktur, modular, dan mudah dipelihara. OOP digunakan secara luas karena mendukung pengembangan perangkat lunak yang lebih kompleks dengan cara yang lebih efisien.

Konsep enkapsulasi menekankan pentingnya melindungi data dalam sebuah objek dengan membatasi akses langsung dari luar objek melalui akses modifier seperti `private` dan `public`. Dengan enkapsulasi, kita bisa memastikan bahwa data sensitif hanya bisa diubah melalui metode tertentu, yang membantu menjaga integritas data dan meminimalkan bug.

Selanjutnya, kita juga mempelajari inheritance (pewarisan), polymorphism (polimorfisme), dan abstraction (abstraksi). Inheritance memungkinkan kita untuk mewarisi sifat dan perilaku dari kelas induk ke kelas turunan, sehingga menghindari duplikasi kode. Polymorphism memungkinkan satu metode memiliki berbagai bentuk atau implementasi yang berbeda, tergantung pada konteksnya. Abstraksi membantu kita menyembunyikan detail kompleks dan hanya menampilkan informasi penting kepada pengguna. Ketiga konsep ini bersama-sama menjadikan OOP sebagai paradigma yang kuat dan fleksibel dalam pengembangan perangkat lunak.

Daftar Pustaka

1. <https://docs.oracle.com/javase/tutorial/>
2. <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
3. Weiss, Mark Allen. Data Structures and Algorithm Analysis in Java. Pearson, 2013.
4. and Algorithms in Java. Sams Publishing, 2002.
5. Sedgewick, Robert, and Kevin Wayne. Algorithms. Addison-Wesley Professional, 2011.
6. Goodrich, Michael T., Roberto Tamassia, and Michael H. Goldwasser. Data Structures and Algorithms in Java. Wiley, 2014.
7. Horstmann, Cay S. Big Java: Early Objects. Wiley, 2016.
8. Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. MIT Press, 2009.